

LAB: Implement SQL Functions: Implement various types of functions in SQL with examples, including String functions, Number Functions, Date Functions and Aggregate Functions.

SQL Functions: SQL functions accept input, perform an operation, and return a result.

The following examples use a STUDENT table.

Student Table

```
CREATE TABLE STUDENT (  
    Student_ID INT,  
    Student_Name VARCHAR(30),  
    Class VARCHAR(10),  
    Marks INT,  
    Date_of_Birth DATE  
);
```

```
INSERT INTO STUDENT VALUES  
(1, 'Rahul Sharma', '10A', 85, '2008-05-15'),  
(2, 'Priya Singh', '10A', 92, '2008-11-20'),  
(3, 'Amit Kumar', '10B', 76, '2009-02-10'),  
(4, 'Sneha Gupta', '10B', 88, '2008-08-25'),  
(5, 'Rohit Verma', '10A', 65, '2009-01-30');  
SELECT * FROM STUDENT;
```

PART A: STRING FUNCTIONS: String functions are used to manipulate “text data”.

Function	Purpose
UPPER()	Converts text to uppercase

LOWER() Converts text to lowercase
LENGTH() Finds the number of characters
Function **Purpose**
CONCAT() Joins strings
SUBSTRING() Extracts part of a string

1. **UPPER()** : Converts student names into uppercase.

```
SELECT Student_Name,  
       UPPER(Student_Name) AS Uppercase_Name  
  
FROM STUDENT;
```

Example Output:

Student_Name	Uppercase_Name
Rahul Sharma	RAHUL SHARMA
Priya Singh	PRIYA SINGH
Amit Kumar	AMIT KUMAR
Sneha Gupta	SNEHA GUPTA
Rohit Verma	ROHIT VERMA

2. **LOWER()** : Converts names into lowercase.

```
SELECT Student_Name,  
       LOWER(Student_Name) AS Lowercase_Name  
  
FROM STUDENT;
```

Student_Name	Lowercase_Name
Rahul Sharma	rahul sharma
Priya Singh	priya singh
Amit Kumar	amit kumar
Sneha Gupta	sneha gupta
Rohit Verma	rohit verma

3. **LENGTH()**: Finds the number of characters in a student's name.

```
SELECT Student_Name,
```

```

        LENGTH(Student_Name) AS Name_Length
FROM STUDENT;

```

Student_Name	Length_No
Rahul Sharma	12
Priya Singh	11
Amit Kumar	10
Sneha Gupta	11
Rohit Verma	11

4. CONCAT() : Combines student name and class.

```

SELECT CONCAT(Student_Name, ' - Class ', Class) AS Student_Details
FROM STUDENT;

```

Student_Details
Rahul Sharma - Class 10A
Priya Singh - Class 10A
Amit Kumar - Class 10B
Sneha Gupta - Class 10B
Rohit Verma - Class 10A

5. SUBSTRING() : Extracts the first five characters of the student's name.

```

SELECT Student_Name,
        SUBSTRING(Student_Name, 1, 5) AS First_Five
FROM STUDENT;

```

Student_Name	First_Five
Rahul Sharma	Rahul
Priya Singh	Priya
Amit Kumar	Amit
Sneha Gupta	Sneha
Rohit Verma	Rohit

PART B: NUMBER FUNCTIONS: Number functions perform mathematical operations on numerical values.

Function Purpose

ROUND() Rounds a number

CEIL() Rounds upward

FLOOR() Rounds downward

ABS() Returns absolute value

MOD() Returns remainder

1. **ROUND()** : Rounds the marks after adding a 5% bonus.

```
SELECT Student_Name,  
       Marks,  
       ROUND(Marks * 1.05) AS Bonus_Marks  
FROM STUDENT;
```

Student_Name	Marks	Bonus_Marks
Rahul Sharma	85	89
Priya Singh	92	97
Amit Kumar	76	80
Sneha Gupta	88	92
Rohit Verma	65	68

2. **CEIL()** : Rounds a number upward.

```
SELECT Student_Name,  
       Marks / 10.0 AS Marks_Value,  
       CEIL(Marks / 10.0) AS Rounded_Up  
FROM STUDENT;
```

Student_Name	Marks_Value	Rounded_Up
Rahul Sharma	8.5000	9
Priya Singh	9.2000	10
Amit Kumar	7.6000	8
Sneha Gupta	8.8000	9
Rohit Verma	6.5000	7

3. FLOOR() : Rounds a number downward.

```
SELECT Student_Name,
       Marks / 10.0 AS Marks_Value,
       FLOOR(Marks / 10.0) AS Rounded_Down
FROM STUDENT;
```

Student_Name	Marks_Value	Rounded_Down
Rahul Sharma	8.5000	8
Priya Singh	9.2000	9
Amit Kumar	7.6000	7
Sneha Gupta	8.8000	8
Rohit Verma	6.5000	6

4. MOD() : Finds the remainder.

```
SELECT Student_Name,
       Marks,
       MOD(Marks, 10) AS Remainder
FROM STUDENT;
```

Student_Name	Marks	Remainder
Rahul Sharma	85	5
Priya Singh	92	2
Amit Kumar	76	6
Sneha Gupta	88	8
Rohit Verma	65	5

5. ABS() : Finds the difference between marks and the target of 80.

```
SELECT Student_Name,
       Marks,
       ABS(Marks - 80) AS Difference
FROM STUDENT;
```

Student_Name	Marks	Difference
Rahul Sharma	85	5
Priya Singh	92	12
Amit Kumar	76	4
Sneha Gupta	88	8
Rohit Verma	65	15

PART C: DATE FUNCTIONS : Date functions are used to work with date values.

Function	Purpose
----------	---------

YEAR()	Extracts year
---------------	---------------

MONTH()	Extracts month
----------------	----------------

DAY()	Extracts day
--------------	--------------

TIMESTAMPDIFF()	Calculating approximate age
------------------------	-----------------------------

CURDATE()	Returns current date
------------------	----------------------

1. YEAR()

```
SELECT Student_Name,  
       Date_of_Birth,  
       YEAR(Date_of_Birth) AS Birth_Year  
FROM STUDENT;
```

Student_Name	Date_of_Birth	Birth_Year
Rahul Sharma	2008-05-15	2008
Priya Singh	2008-11-20	2008
Amit Kumar	2009-02-10	2009
Sneha Gupta	2008-08-25	2008
Rohit Verma	2009-01-30	2009

2. MONTH()

```
SELECT Student_Name,  
       Date_of_Birth,  
       MONTH(Date_of_Birth) AS Birth_Month  
FROM STUDENT;
```

Student_Name	Date_of_Birth	Birth_Month
Rahul Sharma	2008-05-15	5
Priya Singh	2008-11-20	11
Amit Kumar	2009-02-10	2
Sneha Gupta	2008-08-25	8
Rohit Verma	2009-01-30	1

3. DAY()

```
SELECT Student_Name,  
       Date_of_Birth,  
       DAY(Date_of_Birth) AS Birth_Day  
FROM STUDENT;
```

Student_Name	Date_of_Birth	Birth_Day
Rahul Sharma	2008-05-15	15
Priya Singh	2008-11-20	20
Amit Kumar	2009-02-10	10
Sneha Gupta	2008-08-25	25
Rohit Verma	2009-01-30	30

4. TIMESTAMPDIFF()

```
SELECT Student_Name,
       Date_of_Birth,
       TIMESTAMPDIFF(YEAR, Date_of_Birth, CURDATE()) AS Age
FROM STUDENT;
```

Student_Name	Date_of_Birth	Age
Rahul Sharma	2008-05-15	18
Priya Singh	2008-11-20	17
Amit Kumar	2009-02-10	17
Sneha Gupta	2008-08-25	18
Rohit Verma	2009-01-30	17

5. CURDATE()

```
SELECT CURDATE() AS Today;
```

Today
2026-08-25

PART D: AGGREGATE FUNCTIONS : Aggregate functions

perform calculations on multiple rows.

Function	Purpose
COUNT()	Counts records
SUM()	Calculates total
AVG()	Calculates average
MAX()	Finds maximum value
MIN()	Finds minimum value

1. **COUNT()** : Counts the total number of students.

```
SELECT COUNT(*) AS Total_Students
FROM STUDENT; Output:
```

```
+-----+
| Total_Students |
+-----+
|                5 |
+-----+
```

2. **SUM()** : Calculates the total marks.

```
SELECT SUM(Marks) AS Total_Marks
FROM STUDENT;
```

Output:

```
+-----+
| Total_Marks |
+-----+
|          406 |
+-----+
```

3. **AVG()** : Calculates the average marks.

```
SELECT AVG(Marks) AS Average_Marks
FROM STUDENT;
```

```
+-----+
| Average_Marks |
+-----+
|          81.2000 |
+-----+
```

4. **MAX()** : Finds the highest marks.

```
SELECT MAX(Marks) AS Highest_Marks
FROM STUDENT;
```

Output:

```
+-----+
| Highest_Marks |
+-----+
|           92 |
+-----+
```

5. **MIN()** : Finds the lowest marks.

```
SELECT MIN(Marks) AS Lowest_Marks
FROM STUDENT; Output:
```

```
+-----+
| Lowest_Marks |
+-----+
|           65 |
+-----+
```

AGGREGATE FUNCTIONS WITH GROUP BY : used to group rows having the same value in one or more columns, usually along with aggregate functions like COUNT(), SUM(), and AVG()

```
SELECT Class,
       COUNT(*) AS Total_Students,
       AVG(Marks) AS Average_Marks
FROM STUDENT
GROUP BY Class;
```

Class	Total_Students	Average_Marks
10A	3	80.6667
10B	2	82.0000

AGGREGATE FUNCTIONS WITH

HAVING(): used to filter groups of rows created by GROUP BY based on a specified condition.

```
SELECT Class,
       AVG(Marks) AS Average_Marks
FROM STUDENT
GROUP BY Class
HAVING AVG(Marks) > 80;
```

Class	Average_Marks
10A	80.6667
10B	82.0000

AGGREGATE FUNCTION ORDERBY():

ORDER BY is used to sort the result of a query in ascending (ASC) or descending (DESC) order.

```
SELECT Student_ID,
       Student_Name,
       Class,
       Marks
FROM STUDENT
ORDER BY Marks DESC;
```

Student_ID	Student_Name	Class	Marks
2	Priya Singh	10A	92
4	Sneha Gupta	10B	88
1	Rahul Sharma	10A	85
3	Amit Kumar	10B	76
5	Rohit Verma	10A	65

Conclusion

Thus, various SQL functions were implemented using the STUDENT table:

- **String Functions** – UPPER(), LOWER(), LENGTH(), CONCAT(), SUBSTRING()
- **Number Functions** – ROUND(), CEIL(), FLOOR(), MOD(), ABS()
- **Date Functions** – YEAR(), MONTH(), DAY(), CURDATE()
- **Aggregate Functions** – COUNT(), SUM(), AVG(), MAX(), MIN()
- **GroupBy(), Having(), OrderBy()**